

FORCEBIT TICKETING

Een support ticketing applicatie met een gelaagde architectuur

Alex Delvoye

alexdelvoye@outlook.be

Promotor: Emlin Oguz Inci

Co-promotor: Sacha De Maere (Forcebit)

Hogeschool Gent, Valentin Vaerwyckweg 1, 9000 Gent

Samenvatting

Forcebit Ticketing is een proof of concept voor een centrale supportworkflow. De applicatie brengt klantvragen, antwoorden, statussen en bijlagen samen in tickets. De focus ligt op een onderhoudbare opbouw: een gelaagde backend, duidelijke domeinregels en een frontend met herbruikbare schermlogica.

Keuzerichting: Programmeren

Sleutelwoorden: Ticketing, ASP.NET Core, React Native, MySQL, onderhoudbaarheid

Broncode: https://github.com/alexdelvoye/GraduaatsProef_TicketingSystem.git

1. Probleem

Bij Forcebit verliep klantcommunicatie vooral via telefoon en e-mail. Daardoor raakten antwoorden, screenshots, bijlagen en statussen verspreid over verschillende kanalen. De vraag werd daarom hoe ik een onderhoudbare ticketingapplicatie kon bouwen waarin klant en administrator dezelfde duidelijke supportworkflow volgen.



2. Oplossing

Elke supportvraag wordt centraal bewaard als ticket. De eerste beschrijving van de klant wordt meteen het eerste bericht in de conversatie. Daarna kunnen klant en administrator antwoorden en bijlagen toevoegen zolang het ticket open blijft.

Workflow

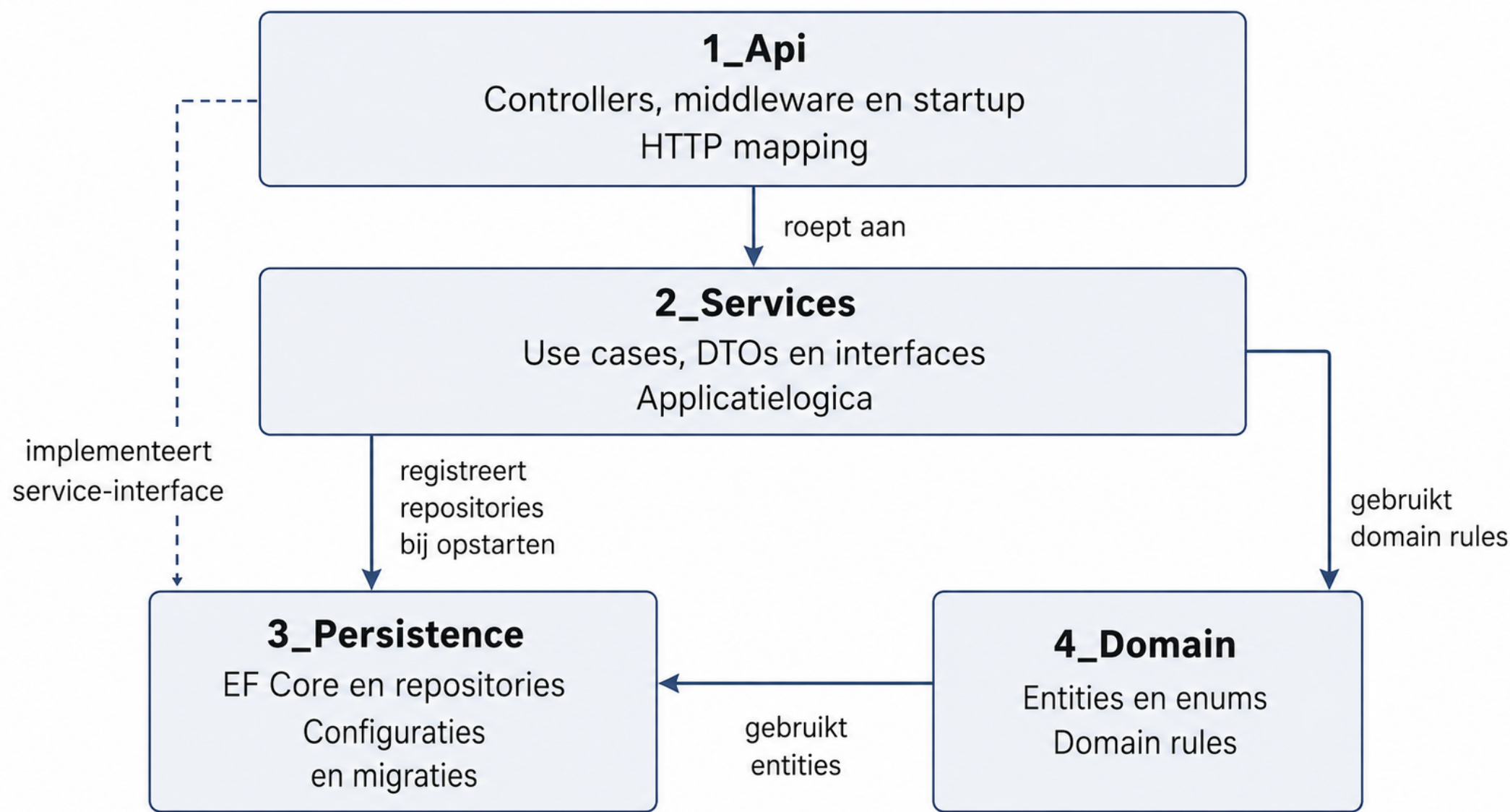
- New: support heeft nog niet geantwoord.
 - Open: de conversatie loopt of het ticket werd heropend.
 - Closed: het ticket is afgehandeld.
- Een heropend ticket wordt opnieuw Open, niet New, omdat het al een geschiedenis heeft.

3. Gelaagde backend

De backend is gebouwd met ASP.NET Core en opgesplitst in vier lagen:

- **API:** HTTP-verzoeken, JWT-claims en requestdata.
- **Services:** use cases zoals antwoorden en statussen wijzigen.
- **Persistence:** EF Core, repositories, configuraties en migraties.
- **Domain:** entiteiten, enums en domeinregels.

Deze verdeling houdt controllers dun. Toegang, statusovergangen en antwoorden op gesloten tickets worden niet in schermen of controllers beslist, maar centraal bewaakt door services en domeinregels.



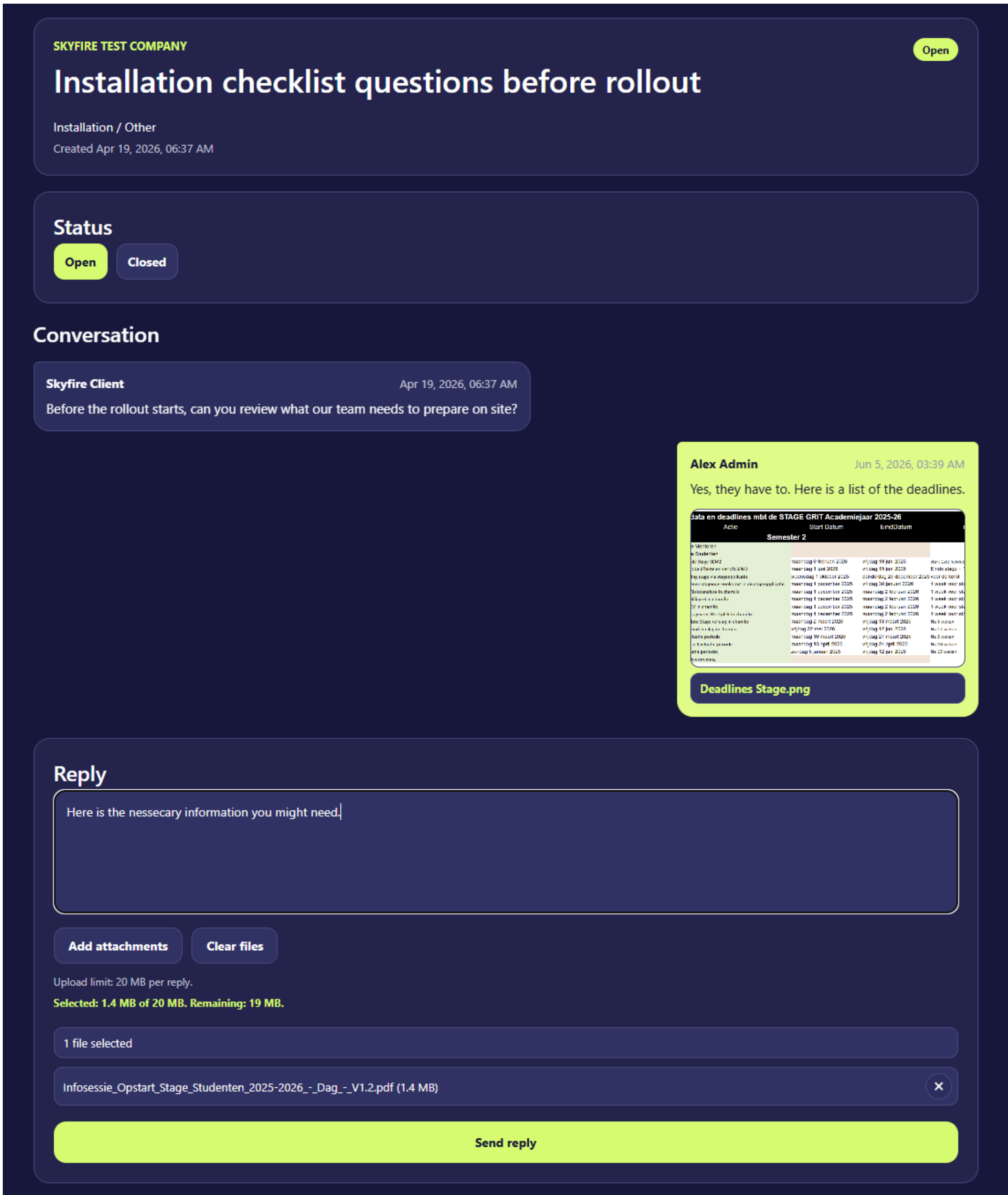
Figuur 1: Backendlagen en belangrijkste afhankelijkheden.

4. Frontend en gebruiksgemak

De frontend is gebouwd met Expo React Native. Schermen tonen de pagina's, hooks beheren gedrag, API-modules communiceren met de backend en componenten houden de UI herbruikbaar. Dezelfde paginatiehook wordt gebruikt voor gesprekken, klanten en ticketgroepen.

Belangrijke UI-keuzes

- zoeken, filters, statusgroepen en paginatie;
- chatbubbels met onderscheid tussen klant en admin;
- preview voor PNG/JPG-afbeeldingen;
- uploadfeedback voor de 20 MB selectielimiet;
- e-mailnotificaties via Brevo bij adminacties.



Figuur 2: Ticketdetail met chatbubbels en afbeeldingspreview.

5. Onderhoudbaarheid

De belangrijkste technische keuze is Single Responsibility. Controllers behandelen HTTP, services voeren workflows uit, repositories halen data op en domeinregels bewaken businessregels. In de frontend geldt dezelfde gedachte: schermen, hooks, API-modules, validatie en componenten blijven gescheiden.

6. Resultaat en vervolg

Het resultaat is een werkend proof of concept voor een realistische supportworkflow: registreren, inloggen, tickets aanmaken, antwoorden, bijlagen toevoegen, zoeken, filteren, pagineren, sluiten, heropenen en notificeren.

Logische vervolgstappen

- productiegerichte opslag voor bijlagen, zoals object storage;
- administrators koppelen aan categorieën, onderwerpen of tickets;
- server-side paginatie en automatische tests;
- verdere uitwerking richting mobiele app.